

The OpenCV Image Processing Pipeline

System Architecture Documentation

1. Introduction

Shape and object recognition does not always require a trained neural network. A well-structured chain of classical image-processing operations can reliably detect and classify geometric shapes in a controlled input image. This report walks through such a pipeline: a linear, five-stage OpenCV processing core wrapped by an input stage and an output stage, with a visualization layer that renders the result back to the user.

Figure 1 shows the complete architecture. An image enters the system through the User and Image Input stages, passes through the OpenCV Processing core, and exits through Visualization and Output.

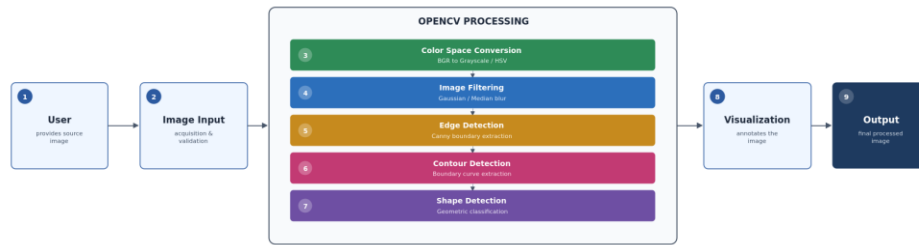


Figure 1. End-to-end architecture of the OpenCV image processing pipeline. The five numbered sub-stages inside the core box execute strictly in sequence.

2. User

The pipeline begins with the user, who supplies the source image to be analyzed. This is the only point of external interaction with the system; every later stage operates automatically without further input.

3. Image Input

The provided image is read into the system, checked for a valid format and file integrity, and loaded into memory as a matrix of pixel values that OpenCV functions can operate on directly.

4. OpenCV Processing

This is the core of the pipeline. The raw image is transformed step by step into structured, analyzable data through five sequential operations, illustrated in Figure 2.

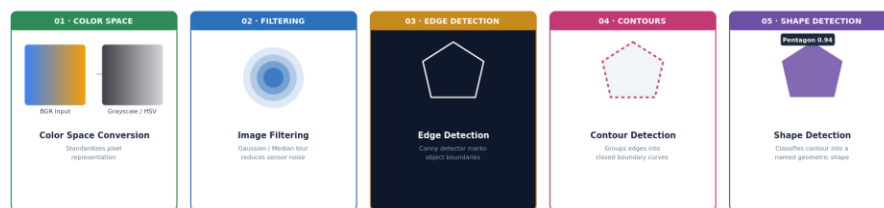


Figure 2. Visual effect of each OpenCV processing sub-stage on the working image.

4.1 Color Space Conversion

The image is converted from BGR to grayscale or HSV, simplifying pixel data and standardizing it for consistent downstream analysis. **Example:** `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`

4.2 Image Filtering

Smoothing filters, such as Gaussian or median blur, remove sensor noise and minor artifacts, preventing false detections in later stages. **Example:** `cv2.GaussianBlur(image, (5, 5), 0)`

4.3 Edge Detection

The Canny edge detector identifies sharp intensity changes, marking the boundaries of every object within the image. **Example:** `cv2.Canny(image, 50, 150)`

4.4 Contour Detection

Connected edge pixels are grouped into continuous contours, each representing the outline of a single distinct object. **Example:** `cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`

4.5 Shape Detection

Each contour is approximated to a polygon and classified, for example as a triangle, rectangle, or circle, based on its vertex count and geometric properties. **Example:** `cv2.approxPolyDP(contour, epsilon, True)`

5. Visualization

Detected shapes, contours, and their labels are drawn back onto the original image using `drawContours` and `putText`, giving the user a clear visual representation of the analysis results before the final output is produced.

6. Output

The final annotated image, along with optional metadata such as shape counts, labels, or coordinates, is returned to the user. This completes the pipeline and closes the loop from raw input to interpreted result.

7. Pipeline Summary

Table 1 summarizes every stage of the pipeline alongside its representative OpenCV operation and purpose.

Stage	Core Operation	Purpose
Image Input	Read & validate image file	Loads raw pixel data into memory
Color Space Conversion	<code>cvtColor (BGR → Gray/HSV)</code>	Standardizes data for analysis
Image Filtering	<code>GaussianBlur / medianBlur</code>	Suppresses noise before edge work
Edge Detection	<code>Canny(image, t1, t2)</code>	Locates object boundaries
Contour Detection	<code>findContours()</code>	Extracts closed boundary curves
Shape Detection	<code>approxPolyDP()</code> + vertex count	Classifies each contour's shape
Visualization	<code>drawContours / putText</code>	Overlays results on the image
Output	<code>imwrite / imshow</code>	Returns final annotated result

Table 1. The pipeline at a glance: stage, OpenCV function, and purpose.

8. Conclusion

This pipeline demonstrates that a carefully ordered sequence of classical OpenCV operations, color conversion, filtering, edge detection, contour extraction, and shape classification, is sufficient to reliably detect and label geometric shapes without any learned model. Its overall accuracy depends on the weakest stage in the chain: poor filtering produces noisy edges, and noisy edges produce broken contours. Understanding each stage's individual role, as documented above, is essential to tuning and debugging the system as a whole.